

Sentinel - Full System Architecture

Neural Nexus · Smart India Hackathon 2026 · Continuous Cyber Risk & Financial Exposure

0. Big picture

Browser app and Python API run on the same local port (8001 here; 8000 belongs to Docker on this machine). Browser never touches code, AI, or math directly - every job is an API call returning JSON, and the page renders only that. One scan ID ties everything together: uploaded copy, findings, risk numbers, project card, and report file.

How Sentinel works (30 seconds)

Paste a GitHub link, ZIP, or folder. Sentinel copies it into a locked practice folder and never touches the live app. AI hacker agents attack only the copy and keep proven hits with file, line, and proof. Each hit gets a 0-100 danger score and a rupee loss from damage times chance. AI planners rank fixes by savings per rupee with cost and hours. A budget optimizer packs maximum savings into your spend and every finding auto-tags to five compliance rulebooks. Dashboards show safety and crores for executives, files and patches for developers, and a one-click report for auditors.

AI agents in Sentinel (ten)

Scan crew - Recon: maps files, routes, libraries, entry points. Exploiter: fires safe payloads for injection, access, and server/client tricks on the copy only, and records proof. Chainer: joins single hits into real impact paths. Coordinator: splits work, dedupes, keeps only proven findings. Brain crew - Risk advisor: answers plain-English questions from live numbers. Fix planner: ranks fixes by savings per rupee with cost and hours. Forecaster: reads history and tells where exposure is heading. Document assistant: answers from compliance documents with cited sources. Defender: drops hidden poison instructions before they reach answers. Budgeter: fits maximum savings inside the budget with returns.

1. Frontend (4 screens, one bundle)

Check-a-project screen holds the GitHub input, optional private-token field with tooltip, Check button, ZIP button, and an injected Folder button backed by a hidden directory picker. Overview shows safety score, money at risk, fix-today count, average fix time computed from per-bug repair hours, proved-by-testing count, trend chart, asset chart, and costliest-problems list. Problems screen is the sortable table (title, technical name, urgency, danger, loss, fix cost) plus a detail drawer (where in code, what was tested, why the AI scored it so, five-part money split). Planner screen orders fixes with cost, hours, savings, and clause tags. A startup script restores the last real scan from browser storage synchronously so the mock generator never fires, refreshes from the latest-results call in the background, clears stale cache when the backend is empty, and toasts every backend step. Old dashboard survives as a backup route.

2. API surface (all calls)

Health check; latest project-plus-findings; project only; GitHub scan (link plus optional token); ZIP upload; folder upload with preserved relative paths; legacy single-file upload and folder intake plus scan-by-ID; risk summary; raw findings; plain-English question; what-if simulation (MFA, patching, segmentation, monitoring, delay days, assumed cost); investment optimize by budget and full curve points; single-framework and full compliance reports; recommendations; document-grounded question; poison inject, poison test, poison clear; document listing; demo-signal ingest; full reset. Errors are JSON with human hints (fictional demo names, shorthand links, private-repo token guidance).

3. Intake flows (all end scanned)

GitHub: link validated (full URL or owner/repo shorthand), optional token injected into the clone URL without logging, shallow clone of depth one into a fresh per-scan folder, language and line counts measured, then scanned. ZIP: saved to temp, extracted to per-scan folder, temp deleted, then scanned. Folder: each file written preserving its relative path under the per-scan folder, top folder becomes the project name, then scanned. Every flow ingests findings into the risk store, rebuilds display findings scaled to computed loss, and sets the project card.

4. Scanning (prove, don't assume)

Preferred path shells to the AI pentest binary against the copied target with the NVIDIA key. Availability gate: binary present, container runtime answering, key set. Its report file is parsed for findings; stdout JSON is a backup. Fallback path reads every code-like file and matches curated patterns across eight bug families with per-family severity, then caps at fifty to avoid spam; a target with zero hits gets deterministic sample findings so the pipeline stays demonstrable. Each kept finding carries id, title, severity, score, category, asset, file path, matched pattern, exploitability, and source tag.

5. Risk math (deterministic, never from the model)

Assets carry business values by criticality tier. Per bug: damage fraction by severity times asset value, plus breach cost (large record count for data bugs, small otherwise, times per-record cost times fraction), plus downtime (hourly cost times 12 critical hours else 4, times fraction), plus regulatory percent plus reputation percent of the sum. Likelihood from severity weight times score-over-ten times exploitability times asset weight times trending boost, clamped to a realistic yearly band; multiple bugs on one asset combine by complement product then rescale. Yearly loss is damage times likelihood with an averaging factor; worst case is the 95th percentile of five thousand simulated lognormal-times-Poisson years with fixed seed. Scores derive from loss and counts; fix time averages per-bug repair hours over an eight-hour day. What-if applies fixed reduction credits per control and a weekly delay penalty, then savings and returns.

6. Display shaping

Backend findings convert to display cards with friendly titles per bug family, technical names, weakness IDs, file locations, six normalized 0-1 signals, validated flags, evidence sentences naming the matched pattern and file, money scaled so parts sum to computed yearly loss, five-part money splits recomputed after scaling, fix cost and hours by severity, one-line fixes, and plain-English risk notes - resorted most-dangerous-first with sequential IDs.

7. AI agents (ten, two crews)

Scan crew of four (mapper, safe attacker, impact chainer, deduplicating coordinator) produces only proven hits. Brain crew of six works proven hits only: advisor answers from live numbers, planner ranks by savings per rupee with cost and hours, forecaster extrapolates history slope, document assistant answers from top retrieved chunks with citations, defender drops poisoned chunks before answering, budgeter packs maximum savings into the spend cap.

8. Model layer

Reasoning model with thinking enabled and long timeout for multi-evidence answers and plans; temperature low, token cap fixed. Guard model plus regex patterns for injection defense. Embeddings prefer the cloud model with local miniature fallback. No training happens anywhere - all models are frozen

APIs.

9. Document search and attack lab

Compliance and risk notes are chunked with overlap, embedded, normalized, and indexed for inner-product search; questions retrieve the top three chunks into cited answers. The lab injects a poison chunk ordering the model to report zero risk and full compliance, then shows the obeying side beside the guarded side that filters it, with detection flags and guard verdicts, plus one-click restore to clean docs.

10. Investment and compliance

Fix catalog of named controls with cost, savings, and percent; exact 0/1 knapsack over lakhs-scaled budget maximizing saved loss, with remaining items ranked by efficiency; curve points across budget steps. Five rulebooks map bug families to control IDs with per-control pass-fail counts, compliance percent, and an averaged governance score plus evidence narrative.

11. State, storage, and honesty rules

In-memory globals hold last findings, project card, and upload ID; history keeps the last hundred risk snapshots. Disk holds per-scan trees, per-scan JSON reports, and the vector index with metadata. Browser holds the last real result for instant paint, auto-cleared when the backend is empty. No seeded demo data anywhere: empty backends yield empty projects, em-dash metrics, and placeholder charts until a real scan completes.

12. Safety properties

Practice-copy-only scanning, read-only intent on sources, container fencing for anything executable, no production address or credential retained, one-time tokens never logged or stored, secrets excluded from version control, live app never contacted by construction.